



Métodos Numéricos Computacionais

Igor dos Reis Gomes

RELATÓRIO TRABALHO 6 – INTEGRAÇÃO NUMÉRICA

Métodos Numéricos Computacionais

Bauru

2025

Bauru
2025
Igor dos Reis Gomes

RELATÓRIO TRABALHO 6 – INTEGRAÇÃO NUMÉRICA

Métodos Numéricos Computacionais

Trabalho teórico-prático apresentado como parte da composição da nota da disciplina de Métodos Numéricos Computacionais – Prof. Douglas Rodrigues.

1. OBJETIVO

O objetivo deste trabalho é aplicar e comparar os métodos de integração numérica para calcular a integral definida de diferentes funções em intervalos pré-determinados. Foram implementadas as regras do Trapézio, 1/3 de Simpson e 3/8 de Simpson para aproximar os valores das integrais. A análise também inclui o cálculo do erro absoluto e relativo para cada método, comparando as aproximações com os valores exatos, para avaliar a precisão e a eficiência de cada método realizado.

2. EXPLICAÇÃO DO CÓDIGO

Neste trabalho, o código foi desenvolvido utilizando a linguagem de programação Python, buscando resolver o problema proposto. Assim, o programa foi organizado de forma modular. O programa é composto por funções para cada método de integração, cálculo de erros, geração de gráficos e uma interface de menu

2.1. Método do Trapézio

Calcula a integral aproximada dividindo a área sob a curva em n trapézios, abaixo mostra o trecho do método implementado.

Trecho 1 - Implementação do método do trapézio

```
1 def metodoTrapezio(func, inicio, fim, n):
2     h = (fim - inicio) / n
3     soma = func(inicio) + func(fim)
4
5     for i in range(1, n):
6         soma += 2 * func(inicio + i * h)
7
8     return h * soma / 2, None
```

Fonte: Elaboração Própria

2.2. Método de $\frac{1}{3}$ de Simpson

Aplica a regra de $\frac{1}{3}$ de Simpson, que requer um número par de subintervalos, para obter uma aproximação mais precisa.

Trecho 2 - Implementação do método de $\frac{1}{3}$ de Simpson

```
1 def metodo13Simpson(func, inicio, fim, n):
2     if n % 2 != 0:
3         return None, "Simpson 1/3 requer número par de subintervalos."
4     h = (fim - inicio) / n
5     soma = func(inicio) + func(fim)
6     for i in range(1, n):
7         peso = 4 if i % 2 != 0 else 2
8         soma += peso * func(inicio + i * h)
9     return h * soma / 3, None
```

Fonte: Elaboração Própria

2.3. Método de $\frac{3}{8}$ de Simpson

Utiliza a regra de $\frac{3}{8}$ de Simpson, que exige um número de subintervalos múltiplo de 3.

Trecho 3 - Implementação do método de $\frac{3}{8}$ de Simpson

```
1 def metodo38Simpson(func, inicio, fim, n):
2     if n % 3 != 0:
3         return None, "Simpson 3/8 requer número de subintervalos múltiplo de 3."
4     h = (fim - inicio) / n
5     soma = func(inicio) + func(fim)
6     for i in range(1, n):
7         if i % 3 == 0:
8             peso = 2
9         else:
10            peso = 3
11            soma += peso * func(inicio + i * h)
12    return 3 * h * soma / 8, None
```

Fonte: Elaboração Própria

2.4. Cálculo dos Erros

Para avaliar a precisão dos métodos, o programa calcula:

- Erro Absoluto: A diferença absoluta entre o valor aproximado e o valor exato.
- Erro Relativo: O erro absoluto dividido pelo valor exato, expresso em porcentagem.

Trecho 4 - Implementação dos erros

```
1 def erroAbs(aprox, real): return abs(real - aprox)
2 def erroRel(aprox, real): return erroAbs(aprox, real) / abs(real)
```

Fonte: Elaboração Própria

2.5. Geração dos Gráficos

Trecho 5 - Implementação dos Gráficos

```
1 # Gráficos
2 def plotGráficos(func_vec, a, b, real, dados, nome_dir):
3     os.makedirs(nome_dir, exist_ok=True)
4     x = np.linspace(a, b, 1000)
5     y = func_vec(x)
6
7     # COMPARAÇÃO APROXIMAÇÕES
8     plt.figure()
9     plt.plot(x, y, label="f(x)", color="black")
10    for metodo, info in dados.items():
11        if 'aprox' in info:
12            plt.axhline(y=info['aprox'], linestyle="--", label=f"{metodo} (≈ {info['aprox']:.4f})")
13    plt.axhline(y=real, color="green", linestyle="-", label=f"Valor real (≈ {real:.4f})")
14    plt.title("Comparação das Aproximações")
15    plt.legend()
16    plt.savefig(f"{nome_dir}/aproximacoes.png")
17    plt.close()
18
19    # ERRO ABSOLUTO
20    plt.figure()
21    metodos = [m for m in dados.keys() if "erro_abs" in dados[m]]
22    erros_abs = [dados[m]["erro_abs"] for m in metodos]
23    plt.bar(metodos, erros_abs, color="tomato")
24    plt.title("Erro Absoluto")
25    plt.savefig(f"{nome_dir}/erro_abs.png")
26    plt.close()
27
28    # ERRO RELATIVO
29    plt.figure()
30    metodos_rel = [m for m in dados.keys() if "erro_rel" in dados[m]]
31    erros_rel = [dados[m]["erro_rel"] for m in metodos_rel]
32    plt.bar(metodos_rel, erros_rel, color="skyblue")
33    plt.title("Erro Relativo")
34    plt.savefig(f"{nome_dir}/erro_rel.png")
35    plt.close()
```

Fonte: Elaboração Própria

Nesta parte, para cada função selecionada, são gerados três tipos de gráficos distintos para visualizar os resultados da integração numérica. O primeiro gráfico plota a curva da função e a compara com linhas horizontais que representam o valor real da integral e as aproximações de cada método implementado. Para garantir que a curva da função seja desenhada suavemente, é utilizado um vetor

com mil pontos no intervalo de integração. Além disso, são criados dois gráficos de barras para facilitar a comparação da precisão entre os métodos. O primeiro exhibe o Erro Absoluto e o segundo, o Erro Relativo, onde cada barra representa um dos métodos de integração.

2.6. Interface

Para o funcionamento do programa, a função menu implementa uma interface interativa em linha de comando, que permite ao usuário escolher qual das três funções matemáticas deseja integrar. Ao selecionar uma opção, o programa calcula as aproximações utilizando os métodos do Trapézio, 1/3 de Simpson e 3/8 de Simpson e exhibe os resultados em uma tabela formatada diretamente no terminal. Essa tabela apresenta a aproximação, o erro absoluto, o erro relativo, o número de subintervalos e o tempo de execução para cada método. Além disso, para a função escolhida, são gerados e salvos automaticamente os gráficos de comparação dos resultados.

Trecho 9 - Implementação da Interface no terminal

```
1 def menu():
2     os.system("cls" if os.name == "nt" else "clear")
3     while True:
4         print("\n===== TRABALHO 6 - INTEGRAÇÃO NUMÉRICA =====\n")
5         print("1. f(x) = x² + 2x + 1      [0, 4]")
6         print("2. f(x) = sin(x)                [0, π]")
7         print("3. f(x) = e^x                      [0, 2]")
8         print("4. Sair")
9         escolha = input("→ ")
10
11         if escolha == '1':
12             os.system("cls" if os.name == "nt" else "clear")
13             resolveIntegral(g1, g1_vec, 0, 4, integraisExatas["g1"], "f(x) = x² + 2x + 1", "saida_funcao1")
14         elif escolha == '2':
15             os.system("cls" if os.name == "nt" else "clear")
16             resolveIntegral(g2, g2_vec, 0, math.pi, integraisExatas["g2"], "f(x) = sin(x)", "saida_funcao2")
17         elif escolha == '3':
18             os.system("cls" if os.name == "nt" else "clear")
19             resolveIntegral(g3, g3_vec, 0, 2, integraisExatas["g3"], "f(x) = e^x", "saida_funcao3")
20         elif escolha == '4':
21             print("Encerrando o programa.")
22             break
23         else:
24             print("Opção inválida. Tente novamente.")
```

Fonte: Elaboração Própria

3. ANÁLISE E DISCUSSÃO

Nesta seção, os resultados obtidos para cada função são apresentados e comparados. A análise focará em qual método forneceu a aproximação mais precisa para cada função, com base nos erros absoluto e relativo. Serão utilizadas as tabelas geradas pelo programa e os gráficos para visualizar e discutir o desempenho de cada método. Segue abaixo as funções utilizadas para calcular as integrais.

1. $f(x) = x^2 + 2x + 1$ no intervalo $[0, 4]$
2. $f(x) = \sin(x)$ no intervalo $[0, \pi]$
3. $f(x) = e^x$ no intervalo $[0, 2]$

Com base nesses dados, segue abaixo os resultados obtidos.

Tabela 1 - Resultados obtidos (função 1)

Método	Aproximação	Erro Absoluto	Erro Relativo	Subintervalos	Tempo Execução
Trapézio	41.44	11.9	22.30%	10	2.84e-05 s
Simpson 1/3	41.3333	12	22.50%	10	7.90e-06 s
Simpson 3/8	41.3333	12	22.50%	6	5.00e-06 s

Fonte: Elaboração Própria

Tabela 2 - Resultados obtidos (função 2)

Método	Aproximação	Erro Absoluto	Erro Relativo	Subintervalos	Tempo Execução
Trapézio	1.98352	0.0165	0.82%	10	1.37e-05 s
Simpson 1/3	2.00011	0.00011	0.01%	10	5.70e-06 s
Simpson 3/8	2.00201	0.00201	0.10%	6	3.30e-06 s

Fonte: Elaboração Própria

Tabela 3 - Resultados obtidos (função 3)

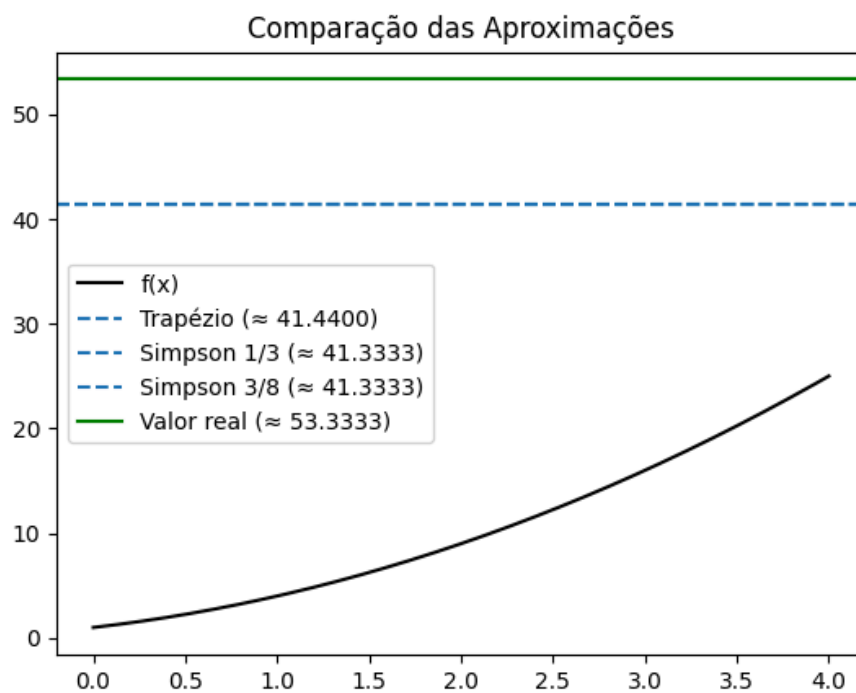
Método	Aproximação	Erro Absoluto	Erro Relativo	Subintervalos	Tempo Execução
Trapézio	6.41034	0.0213	0.33%	10	1.28e-05 s
Simpson 1/3	6.38911	5.65e-05	0.00%	10	4.90e-06 s
Simpson 3/8	6.39002	0.000961	0.02%	6	2.70e-06 s

Fonte: Elaboração Própria

Com base nas tabelas geradas, observa-se que, para as funções analisadas, o método de Simpson 1/3 geralmente apresentou a maior precisão (menores erros relativos), seguido pelo Simpson 3/8 e, por fim, pelo Trapézio. Em termos de tempo de execução, os métodos de Simpson foram consistentemente mais rápidos.

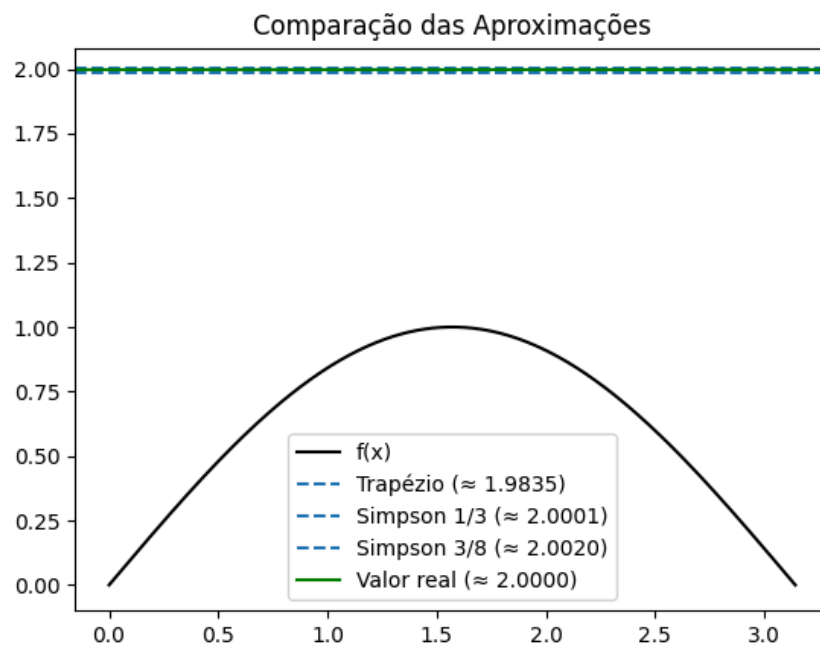
Para análise final, segue os gráficos obtidos:

Figura 1 - Aproximações (função 1)



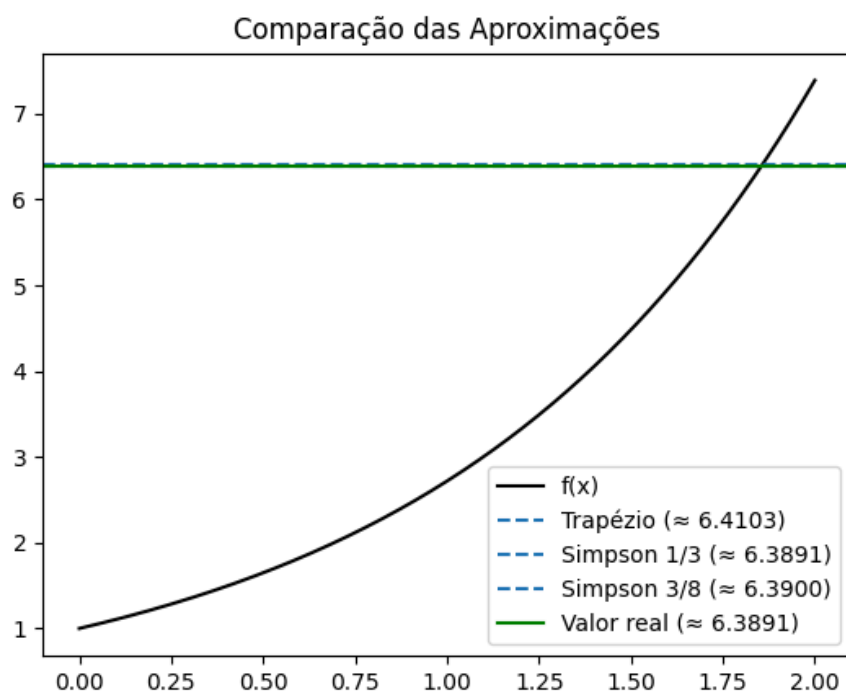
Fonte:Elaboração Própria utilizando Matplotlib

Figura 2 - Aproximações (função 2)



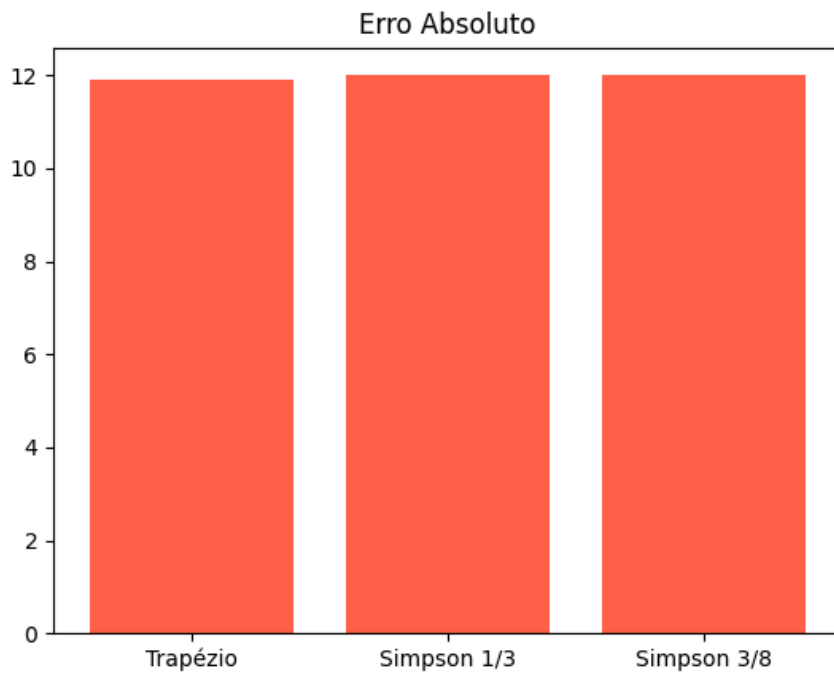
Fonte:Elaboração Própria utilizando Matplotlib

Figura 3 - Aproximações (função 3)



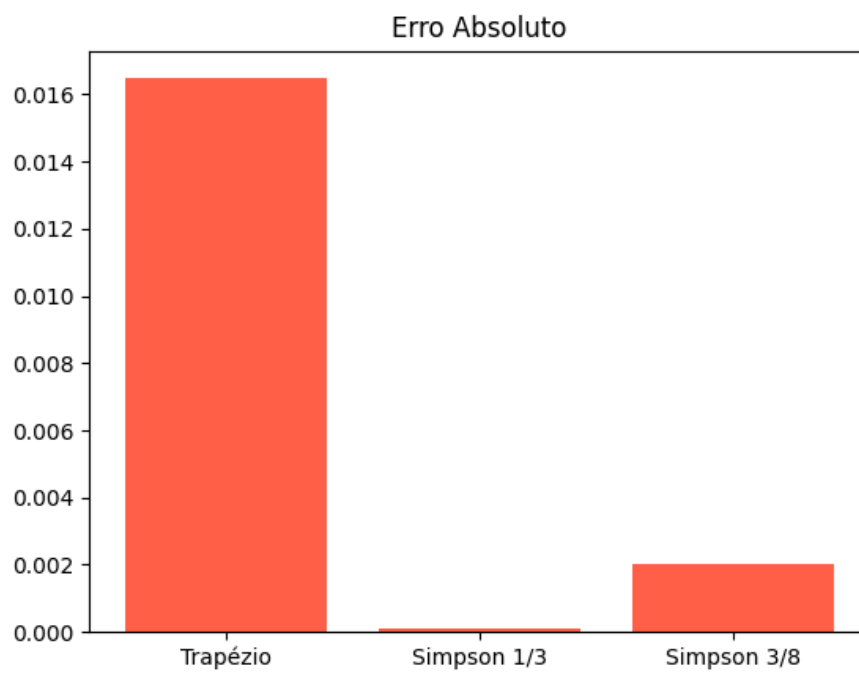
Fonte:Elaboração Própria utilizando Matplotlib

Figura 4 - Erro Absoluto (função 1)



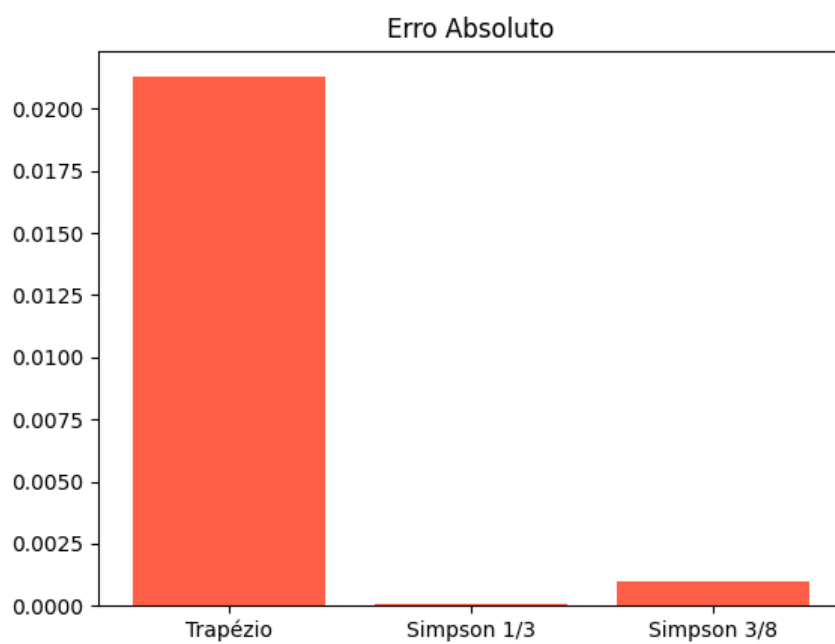
Fonte:Elaboração Própria utilizando Matplotlib

Figura 5 - Erro Absoluto (função 2)



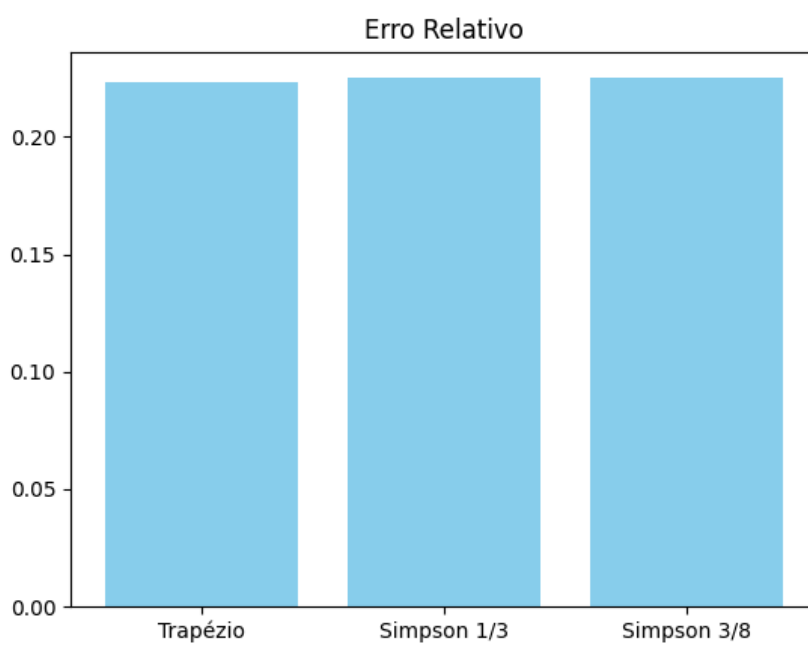
Fonte:Elaboração Própria utilizando Matplotlib

Figura 6 - Erro Absoluto (função 3)



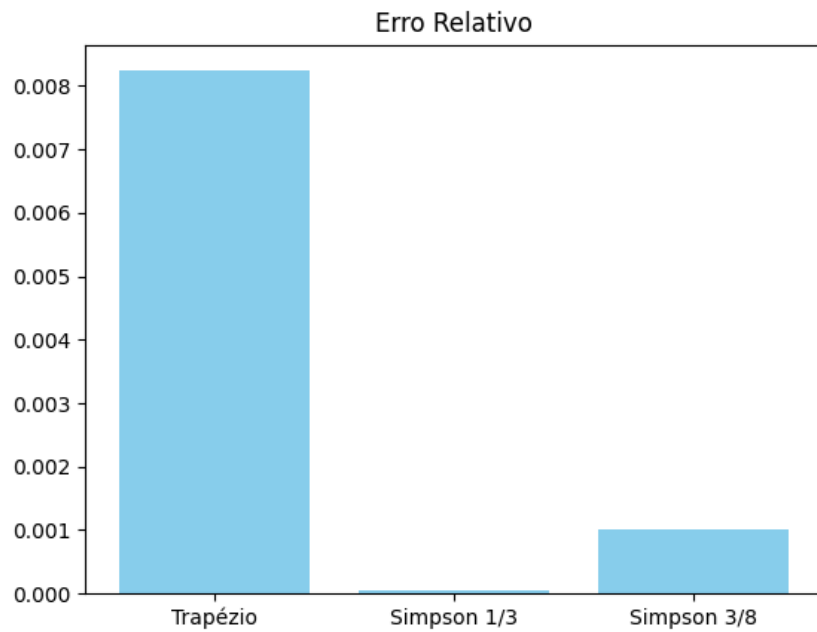
Fonte:Elaboração Própria utilizando Matplotlib

Figura 7 - Erro Relativo (função 1)



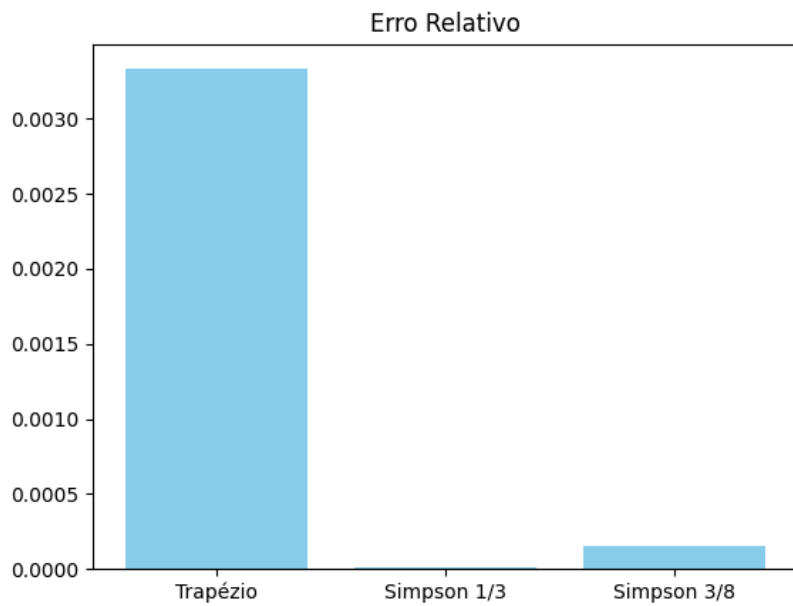
Fonte:Elaboração Própria utilizando Matplotlib

Figura 8 - Erro Relativo (função 2)



Fonte:Elaboração Própria utilizando Matplotlib

Figura 9 - Erro Relativo (função 3)



Fonte:Elaboração Própria utilizando Matplotlib

Os gráficos demonstram que o método de Simpson 1/3 tende a ser o mais preciso, com suas aproximações mais próximas do valor real e os menores erros absoluto e relativo. O método de Simpson 3/8 também apresenta boa precisão,

superando o Trapézio na maioria dos casos. Por outro lado, o método do Trapézio exibe os maiores erros, indicando menor aproximação.

4. CONCLUSÃO

Neste trabalho, foi realizada uma comparação entre os métodos de integração numérica Trapézio, Simpson 1/3 e Simpson 3/8. Observou-se que o método de Simpson 1/3 consistentemente apresentou a maior precisão, resultando nos menores erros absolutos e relativos para as funções analisadas. O método de Simpson 3/8 também demonstrou boa acurácia, geralmente superando o Trapézio, e este demonstrou ser o menos preciso, exibindo os maiores erros.

Conclui-se que, para obter maior precisão em cálculos de integrais, métodos de ordem superior como os de Simpson são preferíveis. Contudo, a escolha ideal do método depende da função específica a ser integrada e do nível de precisão exigido para a aplicação.